If-Else Statements in Python

- **Branching Statements Overview**:
  - An **If-Else Statement** (or simply **If Statement**) is used to control the flow of code.
  - It executes a block of code only if a given logical condition is met.
  - If the condition is true, a specific code block is executed; otherwise, the program skips to the next block.

- **Basic Syntax**:

```python
if condition:
    code_block
else:
    alternative_code_block
```

- Python evaluates the logical expression (condition). If True, the code block executes. If False, it moves to the else block.

- **Indentation**:
  - All lines within the if or else blocks must have the same level of indentation, typically four spaces.

Extended If-Else and Elif Statements

- **Extended If-Else**:
  - You can add multiple conditions using elif (short for else-if).
  - Python checks each condition in sequence, and once it finds a True condition, it executes the corresponding block and exits.

- **Syntax**:

```python
if condition_1:
    code_block_1
elif condition_2:
    code_block_2
elif condition_3:
    code_block_3
else:
    default_code_block
```

- **Key Points**:
  - The elif statement allows checking additional conditions.
  - You can have as many elif statements as needed, but only one else statement.
  - Once a True condition is found, no further conditions are evaluated.

Example - Thermostat Function

- **Task**: Write a function my_thermo_stat(temp, desired_temp) that adjusts a thermostat based on the temperature difference.
- **Functionality**:
  - If the current temperature is more than 5 degrees lower than the desired temperature, return "Heat".
  - If it’s more than 5 degrees higher, return "AC".
  - Otherwise, return "Off".
- **Code Example**:

```python
def my_thermo_stat(temp, desired_temp):
    if temp < desired_temp - 5:
        return 'Heat'
    elif temp > desired_temp + 5:
        return 'AC'
    else:
        return 'Off'
```

- **Sample Usage**:

```python
print(my_thermo_stat(65, 75))  # Output: Heat
print(my_thermo_stat(75, 65))  # Output: AC
print(my_thermo_stat(65, 63))  # Output: Off
```

Ternary Operators in Python

- **Definition**: Ternary operators, also known as conditional expressions, provide a concise way to write simple if-else logic in a single line of code.
- **Syntax**:

```python
expression_if_true if condition else expression_if_false
```

- **Example**:

```python
is_student = True
person = 'student' if is_student else 'not student'
print(person)  # Output: student
```

- **Benefits**:
  - Simplifies code for basic conditionals.
  - Useful for making code more concise and readable.
  - Often employed in list comprehensions and similar constructs.

Equivalent Block for Ternary Operator

- **Ternary Operator Equivalent in Python**:

The example from the previous slide can be expressed in a more traditional if-else structure:

```python
is_student = True
if is_student:
    person = 'student'
else:
    person = 'not student'
print(person)  # Output: student
```

- **Comparison**:
  - The ternary operator offers a more compact representation, reducing the number of lines and increasing readability for simple conditionals.
  - The traditional if-else block is better suited for more complex conditional logic.